Explications générales sur le Thème 4 : Preuves en déduction naturelle

1. Objet du thème
Le Thème 4 change de registre par rapport aux thèmes précédents : il ne s'agit plus de calculer une valeur de vérité ou de mettre une formule en forme normale, mais de **construire une preuve** pas à pas à partir d'hypothèses, en utilisant uniquement les **règles de déduction naturelle** autorisées dans le cours. Le but est de montrer qu'une formule est une conséquence logique d'autres formules, sans table de vérité et sans raisonnement "magique". Tout doit apparaître ligne après ligne.

2. Ce qui est autorisé / interdit dans ce TD
Dans ce thème, en général, le sujet précise :
• utiliser seulement les règles d'**introduction** et d'**élimination** des connecteurs (∧, ∨, ⇒, ¬, parfois ↔) ;
• **ne pas utiliser** le raisonnement par l'absurde global (du type "on suppose ¬F, on arrive à ⊥, donc F") si ce n'est pas explicitement demandé ;
• **ne pas utiliser** le tiers exclu (F ∨ ¬F) ;
• **ne pas utiliser** les règles dérivées (contraposée directe, double négation automatique, etc.), sauf si elles ont été construites dans le cours.
Cela veut dire qu'il faut travailler "à la main", avec les règles de base, et montrer les ouvertures/fermetures de sous-preuves.

3. Règles de base à connaître (format cours)
Ce thème tourne autour d'un petit groupe de règles. Les voici dans le format qu'on attend dans une copie.

(a) Conjonction ∧
• **∧-Introduction** (∧I) : à partir de A et de B, conclure A ∧ B. • **∧-Élimination** (∧E_1 / ∧E_2) : à partir de A ∧ B, conclure A ; à partir de A ∧ B, conclure B. Utilisation typique : quand la conclusion attendue est un ∧, on prouve d'abord la partie gauche, puis la partie droite, puis on fait ∧I.

(b) Implication ⇒
• **⇒-Élimination** (modus ponens) : de A et de A ⇒ B, conclure B. • **⇒-Introduction** : pour montrer A ⇒ B, on ouvre une sous-preuve en supposant A, on démontre B sous cette hypothèse, puis on ferme la sous-preuve et on écrit A ⇒ B. C'est la règle la plus utilisée du TD4. Dès qu'une formule commence par "⇒", c'est presque sûr qu'il faut ouvrir une hypothèse.

(c) Disjonction ∨
• **∨-Introduction gauche** : de A, conclure A ∨ B. • **∨-Introduction droite** : de B, conclure A ∨ B. • **∨-Élimination** (raisonnement par cas) : si on a A ∨ B, et que sous l'hypothèse A on arrive à

C, et que sous l'hypothèse B on arrive aussi à C, alors on conclut C. Cette dernière règle est utilisée dans les exercices "d'énigmes", car elle permet de dire : "si c'est le cas 1 alors C, si c'est le cas 2 alors C, donc dans tous les cas C".

(d) Négation ¬ et contradiction ⊥
• Souvent, dans ce cours, la négation est vue comme une implication vers ⊥ : $\neg A$ ≡ $A \Rightarrow \bot$. • **¬-Introduction** : pour établir ¬A, on ouvre une sous-preuve avec A, on fabrique ⊥, puis on referme et on écrit ¬A. • **⊥-Élimination** : de ⊥, on peut conclure n'importe quelle formule. Attention : ce n'est pas exactement le même schéma que le raisonnement par l'absurde global interdit dans le TD ; ici c'est un usage local de la négation.

4. Méthode de départ (ce qu'il faut faire AVANT d'écrire la preuve)
Avant d'écrire des lignes, il faut regarder la **forme** de la formule à démontrer. C'est le point central du thème : la forme de la conclusion indique la règle à utiliser.

- Si la conclusion est de la forme $A \Rightarrow B$ → il faut faire une **⇒-introduction** → donc on commence par supposer A.
- Si la conclusion est de la forme $A \wedge B$ → il faut faire une **∧-introduction** → donc il faudra démontrer A puis démontrer B.
- Si la conclusion est de la forme $A \vee B$ → il faut faire une **∨-introduction** → donc il suffit de démontrer A ou de démontrer B.
- Si la conclusion est de la forme $\neg A$ → il faut faire une **¬-introduction** → donc on suppose A et on cherche à obtenir ⊥.

Avec ces 4 cas, 90% des exercices du TD4 se débloquent.

5. Structure standard d'une preuve d'implication
Beaucoup de formules du TD4 ont la forme :

```
(H ⇒ …) ⇒ ( … ⇒ … )
```

ou bien

```
(A ⇒ B) ⇒ (¬B ⇒ ¬A)
```

La structure standard est alors :

1. Ouvrir une première hypothèse correspondant au premier ⇒.
2. À l'intérieur, ouvrir une deuxième hypothèse correspondant au deuxième ⇒.
3. Utiliser les hypothèses disponibles pour fabriquer la conclusion du deuxième ⇒.
4. Fermer la deuxième hypothèse → on obtient le deuxième ⇒.
5. Fermer la première hypothèse → on obtient le premier ⇒.

Il faut donc accepter qu'il y aura souvent 2 niveaux de "boîtes" imbriqués.

6. Utilisation correcte des hypothèses

• Une hypothèse ouverte dans une sous-preuve ne vaut que dans cette sous-preuve et dans celles qui sont dedans. • Quand on referme la sous-preuve, l'hypothèse n'est plus disponible. • Donc, pour utiliser une implication déjà présente, il faut être dans une zone où son antécédent est disponible. Sinon, il faut ouvrir une nouvelle sous-preuve pour fabriquer cet antécédent.

7. Schémas utiles (patrons à reconnaître)

Le TD4 contient souvent les schémas suivants :

- **Schéma de la contraposée constructive :** pour montrer `(A ⇒ B) ⇒ (¬B ⇒ ¬A)`, on fait :
  1. supposer `A ⇒ B` ;
  2. supposer `¬B` (c.-à-d. `B ⇒ ⊥`) ;
  3. supposer `A` (pour aller vers ¬A) ;
  4. par modus ponens, de `A` et `A ⇒ B`, on obtient `B` ;
  5. puis, de `B` et `¬B`, on obtient ⊥ ;
  6. on referme pour écrire ¬A ;
  7. on referme pour écrire `¬B ⇒ ¬A` ;
  8. on referme pour écrire la formule entière.

  Ce patron revient souvent.
- **Schéma "je veux un ∧" :** prouver séparément les deux côtés, puis ∧I.
- **Schéma "je veux un ∨" :** prouver un seul côté, puis ∨I.
- **Schéma "j'ai un ∨" et je veux en sortir quelque chose :** faire une ∨E avec deux sous-preuves, une pour chaque cas.

8. Erreurs fréquentes (à corriger dans les copies)

• sauter une étape de sous-preuve (écrire directement `A ⇒ B` sans avoir ouvert A) ;
• utiliser la contraposée sans la prouver (interdit ici) ;
• faire un raisonnement par l'absurde global alors que le sujet ne l'autorise pas ;
• utiliser une hypothèse en dehors de la boîte où elle a été ouverte ;
• mélanger déduction naturelle et table de vérité (ce TD ne veut pas de tables).

9. Résumé opérationnel (à mettre en haut du TD4)

1. Regarder la forme de la conclusion.
2. Choisir la règle d'introduction qui correspond.
3. Ouvrir les sous-preuves nécessaires.
4. Utiliser ∧E, ⇒E, et les hypothèses pour fabriquer la formule voulue.
5. Fermer dans l'ordre inverse.
6. Ne jamais utiliser une règle non autorisée.

10. Phrase de fin à donner aux étudiants
“Pour chaque formule du TD4, la preuve se construit en lisant la forme de la conclusion et en appliquant la règle d’introduction correspondante. Tant que les introductions sont faites au bon moment et que les hypothèses sont bien ouvertes/fermées, la preuve est correcte, même si elle est un peu longue.”

````
````markdown # Déductions naturelles — modèle “Edukera dessiné” (version complète)
````

On fait toujours **dans le même ordre** :

1. **On écrit le but** (ce qu’on veut montrer).
2. **On ouvre** si le but commence par ⇒ ou si on doit faire une négation.
3. **On déroule** (⇒E, ∧E, ∧I, ∨I, ∨E, ¬I).
4. **On ferme** avec les numéros.
5. **On passe au prochain exemple.**

Je te mets **d’abord** les exemples “classiques” (ceux que le prof adore), **puis** je crée d’autres exemples de même style pour t’entraîner.

Exemple 1 (très basique)

But : à partir de `p` et `q`, déduire `p ∧ q`.

```
(1)  p
     hyp.
(2)  q
     hyp.
(3)  p ∧ q
     par ∧I avec (1), (2)
```

On avait les deux morceaux → on colle.

Exemple 2 (le vrai modus ponens)

But : de `p ⇒ q` et `p`, déduire `q`.

```
(1)  p ⇒ q
     hyp.
(2)  p
     hyp.
```
```

```
(3)  q
     par ⇒E avec (1), (2)
```

C’est la forme la plus utilisée dans tout le TD.

Exemple 3 (forme cours)

But : `⊢ (p ∧ q) ⇒ p`

```
(1)  montrons (p ∧ q) ⇒ p
     ┌──────────────────────────────────────┐
(2)  │ supposons h₁ : p ∧ q                 │
     │                                      │
(3)  │ p                                    │
     │   par ∧E₁ avec h₁                    │
     └──────────────────────────────────────┘
(4)  (p ∧ q) ⇒ p
     par ⇒I sur (2)–(3)
```

Schéma à retenir : **conclusion commence par ⇒ → on suppose le truc de gauche**.

Exemple 4 (intro du ∨)

But : `⊢ p ⇒ (p ∨ q)`

```
(1)  montrons p ⇒ (p ∨ q)
     ┌──────────────────────────────────────┐
(2)  │ supposons h₁ : p                     │
     │                                      │
(3)  │ p ∨ q                                │
     │   par ∨I₁ avec h₁                    │
     └──────────────────────────────────────┘
(4)  p ⇒ (p ∨ q)
     par ⇒I sur (2)–(3)
```

On avait `p`, donc on a le droit de faire `p ∨ q`.

Exemple 5 (le modèle du TD)

But : `⊢ p ⇒ (q ⇒ p)`

```
(1)  montrons p ⇒ (q ⇒ p)
     ┌──────────────────────────────────────────┐
(2)  │ supposons h₁ : p                         │
     │   ┌──────────────────────────────────┐   │
(3)  │   │ supposons h₂ : q                 │   │
     │   │                                  │   │
(4)  │   │ p                                │   │
     │   │   reprise de h₁                  │   │
     │   └──────────────────────────────────┘   │
(5)  │ q ⇒ p                                    │
     │   par ⇒I sur (3)–(4)                     │
     └──────────────────────────────────────────┘
(6)  p ⇒ (q ⇒ p)
     par ⇒I sur (2)–(5)
```

Schéma très important : **on ouvre, on ouvre, on ferme, on ferme.**

Exemple 6 (chaînage)

But : `⊢ (p ⇒ q) ⇒ ((q ⇒ r) ⇒ (p ⇒ r))`

```
(1)  montrons (p ⇒ q) ⇒ ((q ⇒ r) ⇒ (p ⇒ r))
     ┌──────────────────────────────────────────────────────┐
(2)  │ supposons h₁ : p ⇒ q                                 │
     │   ┌──────────────────────────────────────────┐       │
(3)  │   │ supposons h₂ : q ⇒ r                     │       │
     │   │   ┌──────────────────────────────┐       │       │
(4)  │   │   │ supposons h₃ : p             │       │       │
     │   │   │                              │       │       │
(5)  │   │   │ q                            │       │       │
     │   │   │   par ⇒E avec h₁, h₃         │       │       │
(6)  │   │   │ r                            │       │       │
     │   │   │   par ⇒E avec h₂, (5)        │       │       │
     │   │   └──────────────────────────────┘       │       │
(7)  │   │ p ⇒ r                                    │       │
     │   │   par ⇒I sur (4)–(6)                     │       │
     │   └──────────────────────────────────────────┘       │
(8)  │ (q ⇒ r) ⇒ (p ⇒ r)                                    │
     │   par ⇒I sur (3)–(7)                                 │
     └──────────────────────────────────────────────────────┘
(9)  (p ⇒ q) ⇒ ((q ⇒ r) ⇒ (p ⇒ r))
     par ⇒I sur (2)–(8)
```

On voit bien les **3 niveaux** comme dans ton image.

Exemple 7 (contraposée)

But : $\vdash (A \Rightarrow B) \Rightarrow (\neg B \Rightarrow \neg A)$

```
(1)  montrons (A ⇒ B) ⇒ (¬B ⇒ ¬A)
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : A ⇒ B                             │
     │   ┌──────────────────────────────────────────┐   │
(3)  │   │ supposons h₂ : ¬B                        │   │
     │   │   ┌──────────────────────────────────┐   │   │
(4)  │   │   │ supposons h₃ : A                 │   │   │
(5)  │   │   │ B                                │   │   │
     │   │   │   par ⇒E avec h₁, h₃             │   │   │
(6)  │   │   │ ⊥                                │   │   │
     │   │   │   par E¬ avec h₂, (5)            │   │   │
     │   │   └──────────────────────────────────┘   │   │
(7)  │   │ ¬A                                       │   │
     │   │   par ¬I sur (4)–(6)                     │   │
     │   └──────────────────────────────────────────┘   │
(8)  │ ¬B ⇒ ¬A                                          │
     │   par ⇒I sur (3)–(7)                             │
     └──────────────────────────────────────────────────┘
(9)  (A ⇒ B) ⇒ (¬B ⇒ ¬A)
     par ⇒I sur (2)–(8)
```

Même technique que tu as aimée.

Exemple 8 (tiers exclu)

But : $\vdash A \vee \neg A$

```
(1)  montrons A ∨ ¬A
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : ¬(A ∨ ¬A)                         │
     │   ┌──────────────────────────────────────────┐   │
(3)  │   │ montrons ¬A                              │   │
     │   │   ┌──────────────────────────────────┐   │   │
(4)  │   │   │ supposons h₂ : A                 │   │   │
(5)  │   │   │ A ∨ ¬A                           │   │   │
     │   │   │   par ∨I₁ avec h₂                │   │   │
(6)  │   │   │ ⊥                                │   │   │
     │   │   │   par E¬ avec h₁, (5)            │   │   │
     │   │   └──────────────────────────────────┘   │   │
(7)  │   │ ¬A                                       │   │
     │   │   par ¬I sur (4)–(6)                     │   │
```

```
     │   └──────────────────────────────────────────────────────┘   │
(8)  │ A ∨ ¬A                                                       │
     │   par ∨I₂ avec (7)                                           │
(9)  │ ⊥                                                            │
     │   par E¬ avec h₁, (8)                                        │
     └──────────────────────────────────────────────────────────────┘
(10) A ∨ ¬A
     par RAA sur (2)–(9)
```

C'est le même dessin que tu voulais 👍

Maintenant : nouveaux exercices du même style

Je te fais 4 de plus, mêmes règles, même style.

Exercice A

But : `⊢ (p ⇒ r) ∧ (q ⇒ r) ∧ (p ∨ q) ⇒ r`

```
(1)  montrons (p ⇒ r) ∧ (q ⇒ r) ∧ (p ∨ q) ⇒ r
     ┌──────────────────────────────────────────────────────────────┐
(2)  │ supposons h₁ : (p ⇒ r) ∧ (q ⇒ r) ∧ (p ∨ q)                   │
     │                                                              │
(3)  │ (p ⇒ r) ∧ (q ⇒ r)                                            │
     │   par ∧E₁ avec h₁                                            │
(4)  │ p ∨ q                                                        │
     │   par ∧E₂ avec h₁                                            │
(5)  │ p ⇒ r                                                        │
     │   par ∧E₁ avec (3)                                           │
(6)  │ q ⇒ r                                                        │
     │   par ∧E₂ avec (3)                                           │
     │   ┌──────────── cas p ────────────┐                          │
(7)  │   │ p                             │ hyp.                     │
(8)  │   │ r                             │ par ⇒E avec (5), (7)     │
     │   └───────────────────────────────┘                          │
     │   ┌──────────── cas q ────────────┐                          │
(9)  │   │ q                             │ hyp.                     │
(10) │   │ r                             │ par ⇒E avec (6), (9)     │
     │   └───────────────────────────────┘                          │
(11) │ r                                                            │
     │   par ∨E avec (4), (7)–(8), (9)–(10)                         │
     └──────────────────────────────────────────────────────────────┘
(12) (p ⇒ r) ∧ (q ⇒ r) ∧ (p ∨ q) ⇒ r
     par ⇒I sur (2)–(11)
```

Exercice B

But : `⊢ (p ⇒ q) ⇒ (¬q ⇒ ¬p)` (c'est la contraposée, version p/q)

```
(1)  montrons (p ⇒ q) ⇒ (¬q ⇒ ¬p)
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : p ⇒ q                             │
     │   ┌──────────────────────────────────────────┐   │
(3)  │   │ supposons h₂ : ¬q                        │   │
     │   │   ┌──────────────────────────────┐       │   │
(4)  │   │   │ supposons h₃ : p             │   │   │   │
(5)  │   │   │ q                            │   │   │   │
     │   │   │   par ⇒E avec h₁, h₃         │   │   │   │
(6)  │   │   │ ⊥                            │   │   │   │
     │   │   │   par E¬ avec h₂, (5)        │   │   │   │
     │   │   └──────────────────────────────┘       │   │
(7)  │   │ ¬p                                       │   │
     │   │   par ¬I sur (4)–(6)                     │   │
     │   └──────────────────────────────────────────┘   │
(8)  │ ¬q ⇒ ¬p                                          │
     │   par ⇒I sur (3)–(7)                             │
     └──────────────────────────────────────────────────┘
(9)  (p ⇒ q) ⇒ (¬q ⇒ ¬p)
     par ⇒I sur (2)–(8)
```

Exercice C

But : `⊢ (p ∧ (q ∨ r)) ⇒ ((p ∧ q) ∨ (p ∧ r))` (c'est la distributivité logique, bien vue en TD)

```
(1)  montrons (p ∧ (q ∨ r)) ⇒ ((p ∧ q) ∨ (p ∧ r))
     ┌──────────────────────────────────────────────────────────┐
(2)  │ supposons h₁ : p ∧ (q ∨ r)                               │
     │                                                          │
(3)  │ p                                                        │
     │   par ∧E₁ avec h₁                                        │
(4)  │ q ∨ r                                                    │
     │   par ∧E₂ avec h₁                                        │

     │   ┌──────────── cas q ────────────┐
(5)  │   │ q                             │ hyp.
(6)  │   │ p ∧ q                         │ par ∧I avec (3), (5)
```

```
(7)  |   | (p ∧ q) ∨ (p ∧ r)          | par ∨I₁ avec (6)
     |   └──────────────────────────┘

     |   ┌──────────── cas r ──────────┐
(8)  |   | r                          | hyp.
(9)  |   | p ∧ r                      | par ∧I avec (3), (8)
(10) |   | (p ∧ q) ∨ (p ∧ r)          | par ∨I₂ avec (9)
     |   └──────────────────────────┘

(11) | (p ∧ q) ∨ (p ∧ r)
     |   par ∨E avec (4), (5)–(7), (8)–(10)
     └──────────────────────────────────────────────────────┘
(12) (p ∧ (q ∨ r)) ⇒ ((p ∧ q) ∨ (p ∧ r))
     par ⇒I sur (2)–(11)
```

Exercice D

But : $\vdash (p \Rightarrow q) \Rightarrow ((r \vee p) \Rightarrow (r \vee q))$

```
(1)  montrons (p ⇒ q) ⇒ ((r ∨ p) ⇒ (r ∨ q))
     ┌──────────────────────────────────────────────────────┐
(2)  | supposons h₁ : p ⇒ q                                 |
     |   ┌──────────────────────────────────────────┐       |
(3)  |   | supposons h₂ : r ∨ p                     |       |

     |   |   ┌──────── cas r ──────────┐
(4)  |   |   | r                      | hyp.
(5)  |   |   | r ∨ q                  | par ∨I₁ avec (4)
     |   |   └──────────────────────┘

     |   |   ┌──────── cas p ──────────┐
(6)  |   |   | p                      | hyp.
(7)  |   |   | q                      | par ⇒E avec h₁, (6)
(8)  |   |   | r ∨ q                  | par ∨I₂ avec (7)
     |   |   └──────────────────────┘

(9)  |   | r ∨ q                      |
     |   |   par ∨E avec (3), (4)–(5), (6)–(8)
     |   └──────────────────────────────────────────┘
(10) | (r ∨ p) ⇒ (r ∨ q)                                    |
     |   par ⇒I sur (3)–(9)                                 |
     └──────────────────────────────────────────────────────┘
(11) (p ⇒ q) ⇒ ((r ∨ p) ⇒ (r ∨ q))
     par ⇒I sur (2)–(10)
```

Mini-règles à recopier en bas

- ⇒I : pour prouver A ⇒ B, on suppose A, on montre B, on ferme.
- ⇒E : de A ⇒ B et de A, on déduit B.
- ∧I : de A et B, on déduit A ∧ B.
- ∧E : de A ∧ B, on déduit A ou B.
- ∨I_1 : de A, on déduit A ∨ B.
- ∨I_2 : de B, on déduit A ∨ B.
- ∨E : de A ∨ B, et (A ⊢ R), (B ⊢ R), on déduit R.
- ¬I : de (suppose A ⊢ ⊥) on déduit ¬A.
- RAA : de (suppose ¬A ⊢ ⊥) on déduit A.

Tu peux prendre n'importe lequel de ces modèles, changer seulement les lettres, et tu as un nouvel exercice "comme Edukera".

::contentReference[oaicite:0]{index=0}

```
# Déduction naturelle – Série d'exercices supplémentaires (style Edukera)

Dans cette partie tu vas voir **3 choses** :
1. **La méthode** (comment on commence une preuve, comment on sait s'il faut "supposons…").
2. **Des exercices nouveaux** (pas ceux du TD, mais même niveau).
3. **La solution écrite exactement comme ton modèle Edukera** (numéros, "supposons", "montrons", boîtes).

L'idée c'est que tu puisses juste **copier/coller** pour tes étudiants et qu'eux voient **le même format à chaque fois**.

---

## 0. Méthode (à apprendre par cœur)

1. **Regarder la forme de la conclusion**
   - si ça commence par `A ⇒ B` → **on suppose A**.
   - si ça commence par `¬A` → **on suppose A** et on cherche `⊥`.
   - si ça commence par `A ∨ B` → on montre un des deux (souvent on a A ou B juste avant).
   - si ça commence par une grosse structure `(A ∧ B) ⇒ ...` → on suppose tout le paquet.
```

````
2. **Dedans, on ne fait que 4–5 règles**
   - `⇒E` (modus ponens)
   - `∧E` (projection)
   - `∧I` (coller deux choses)
   - `∨I₁`, `∨I₂` (mettre dans un OU)
   - parfois `∨E` (deux cas)
   - parfois `¬I` (contradiction)

3. **On ferme** toujours en indiquant les lignes :
   - `par ⇒I sur (k)–(m)`
   - `par ¬I sur (k)–(m)`
   - `par ∨E avec (l), (bloc1), (bloc2)`

4. **Toujours le même dessin** :
   - on ouvre
   - on fait les lignes
   - on referme
   - on passe au dessus

Si tu respectes juste ça, **tous** les exos de ton TD4 passent.

---

## Exercice 1
**Montrer** : `⊢ p ⇒ (p ∧ p)`

> (c'est idiot mais très bon pour montrer ∧I)

```text
(1)  montrons p ⇒ (p ∧ p)
     ┌──────────────────────────────────────┐
(2)  │ supposons h₁ : p                     │
(3)  │ p ∧ p                                │
     │   par ∧I avec (2), (2)               │
     └──────────────────────────────────────┘
(4)  p ⇒ (p ∧ p)
     par ⇒I sur (2)–(3)
````

💡 **Ce qu'on a fait** : conclusion = implication → on suppose le gauche → on fabrique le droit avec ∧I.

Exercice 2

Montrer : `⊢ (p ∧ q) ⇒ (q ∧ p)`

> c'est juste "inverser" la conjonction.
````

```
(1)  montrons (p ∧ q) ⇒ (q ∧ p)
     ┌──────────────────────────────────────┐
(2)  │ supposons h₁ : p ∧ q                 │
(3)  │ p                                    │
     │   par ∧E₁ avec h₁                    │
(4)  │ q                                    │
     │   par ∧E₂ avec h₁                    │
(5)  │ q ∧ p                                │
     │   par ∧I avec (4), (3)               │
     └──────────────────────────────────────┘
(6)  (p ∧ q) ⇒ (q ∧ p)
     par ⇒I sur (2)–(5)
```

🟣 **À retenir** : quand tu veux prouver une conjonction, tu prouves les deux morceaux séparément **puis** tu fais ∧I.

Exercice 3

Montrer : `⊢ (p ⇒ q) ⇒ (p ⇒ (p ∧ q))`

> on a déjà vu un truc proche (`p ⇒ (q ⇒ p)`), là c'est la version "j'ajoute p".

```
(1)  montrons (p ⇒ q) ⇒ (p ⇒ (p ∧ q))
     ┌──────────────────────────────────────────┐
(2)  │ supposons h₁ : p ⇒ q                     │
     │   ┌──────────────────────────────┐       │
(3)  │   │ supposons h₂ : p             │       │
(4)  │   │ q                            │       │
     │   │   par ⇒E avec h₁, h₂         │       │
(5)  │   │ p ∧ q                        │       │
     │   │   par ∧I avec (3), (4)       │       │
     │   └──────────────────────────────┘       │
(6)  │ p ⇒ (p ∧ q)                              │
     │   par ⇒I sur (3)–(5)                     │
     └──────────────────────────────────────────┘
(7)  (p ⇒ q) ⇒ (p ⇒ (p ∧ q))
     par ⇒I sur (2)–(6)
```

🟢 **Méthode** :

- il y a 2 "⇒" dans la conclusion → donc 2 fois "supposons...", l'un dans l'autre.
- à l'intérieur : modus ponens, puis ∧I.

Exercice 4

Montrer : `⊢ (p ∨ q) ⇒ (q ∨ p)`

> Là on doit faire un vrai **∨E** (2 cas).

```
(1)  montrons (p ∨ q) ⇒ (q ∨ p)
     ┌──────────────────────────────────────────────────────┐
(2)  │ supposons h₁ : p ∨ q                                 │
     │                                                      │
     │   ┌──────────── cas p ────────────┐
(3)  │   │ p                             │ hyp.
(4)  │   │ q ∨ p                         │ par ∨I₂ avec (3)
     │   └───────────────────────────────┘
     │   ┌──────────── cas q ────────────┐
(5)  │   │ q                             │ hyp.
(6)  │   │ q ∨ p                         │ par ∨I₁ avec (5)
     │   └───────────────────────────────┘
(7)  │ q ∨ p                             │
     │   par ∨E avec (2), (3)–(4), (5)–(6)
     └──────────────────────────────────────────────────────┘
(8)  (p ∨ q) ⇒ (q ∨ p)
     par ⇒I sur (2)–(7)
```

🟠 **À retenir** :

1. tu écris le cas `p ...`
2. tu écris le cas `q ...`
3. tu fermes avec `∨E` en citant les deux blocs.

Exercice 5

Montrer : `⊢ ¬p ⇒ (p ⇒ q)`

> logique : “si p est faux, alors ‘p ⇒ q’ est vrai”.

```
(1)  montrons ¬p ⇒ (p ⇒ q)
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : ¬p                                │
     │   ┌──────────────────────────────────────┐       │
```

```
(3)  |   | supposons h₂ : p                    |   |
(4)  |   | ⊥                                    |   |
     |   |   par E¬ avec h₁, h₂                |   |
(5)  |   | q                                    |   |
     |   |   par ⊥E sur (4)                    |   |
     |   └──────────────────────────────────────┘   |
     | p ⇒ q                                        |
     |   par ⇒I sur (3)–(5)                        |
     └──────────────────────────────────────────────┘

(7)  ¬p ⇒ (p ⇒ q)
     par ⇒I sur (2)–(6)
```

🔵 **Remarques** :

- on a utilisé la petite règle “de ⊥ tu peux tout déduire” (explosion).
- c’est exactement la forme qu’on utilise pour dire “si c’est impossible, alors l’implication est vraie”.

Exercice 6

Montrer : $\vdash (p \Rightarrow q) \land (p \Rightarrow r) \Rightarrow (p \Rightarrow (q \land r))$

> très classique (se ramène à faire 2 modus ponens).

```
(1)  montrons (p ⇒ q) ∧ (p ⇒ r) ⇒ (p ⇒ (q ∧ r))
     ┌──────────────────────────────────────────────────────┐
(2)  | supposons h₁ : (p ⇒ q) ∧ (p ⇒ r)                    |
     |                                                     |
(3)  | p ⇒ q                                               |
     |   par ∧E₁ avec h₁                                   |
(4)  | p ⇒ r                                               |
     |   par ∧E₂ avec h₁                                   |
     |   ┌─────────────────────────────────────────┐       |
(5)  |   | supposons h₂ : p                        |       |
(6)  |   | q                                       |       |
     |   |   par ⇒E avec (3), (5)                  |       |
(7)  |   | r                                       |       |
     |   |   par ⇒E avec (4), (5)                  |       |
(8)  |   | q ∧ r                                   |       |
     |   |   par ∧I avec (6), (7)                  |       |
     |   └─────────────────────────────────────────┘       |
(9)  | p ⇒ (q ∧ r)                                         |
     |   par ⇒I sur (5)–(8)                                |
     └──────────────────────────────────────────────────────┘
(10) (p ⇒ q) ∧ (p ⇒ r) ⇒ (p ⇒ (q ∧ r))
     par ⇒I sur (2)–(9)
```

🟣 **Structure** :

- 1er niveau : je sors les 2 implications.
- 2e niveau : je suppose p.
- dedans : 2 modus ponens → puis ∧I → puis on ferme.

Exercice 7 (un peu comme ton “montrons A ∨ ¬A” mais plus simple)

Montrer : `⊢ ¬¬p ⇒ p`

```
(1)  montrons ¬¬p ⇒ p
     ┌──────────────────────────────────────────┐
(2)  │ supposons h₁ : ¬¬p                       │
     │   ┌──────────────────────────────────┐   │
(3)  │   │ supposons h₂ : ¬p                │   │
(4)  │   │ ⊥                                │   │
     │   │   par E¬ avec h₁, h₂             │   │
     │   └──────────────────────────────────┘   │
(5)  │ p                                        │
     │   par ¬I sur (3)–(4)                     │
     └──────────────────────────────────────────┘
(6)  ¬¬p ⇒ p
     par ⇒I sur (2)–(5)
```

🟢 **C’est le même pattern** que ton grand exemple :

- on suppose la négation,
- on obtient ⊥ avec la double négation,
- on ferme en ¬I.

Comment toi tu peux en inventer d’autres

Tu prends **une règle d’intro** et tu construis l’exo à l’envers :

1. **Je veux finir par ⇒I** → donc mon énoncé sera une implication.
2. **Je veux finir par ∧I** → donc mon énoncé sera une conjonction.
3. **Je veux faire ∨E** → donc mon énoncé commencera par un OU + deux implications.
4. **Je veux faire ¬I** → donc mon énoncé sera une négation.

Exemples d’énoncés que tu peux donner aux étudiants (à faire EXACTEMENT comme ci-dessus) :

- `⊢ p ⇒ p`

- `⊢ p ⇒ (q ⇒ (p ∧ q))`
- `⊢ (p ∧ r) ⇒ (r ∧ p)`
- `⊢ (p ⇒ q) ⇒ ((q ⇒ ⊥) ⇒ (p ⇒ ⊥))`
- `⊢ (p ⇒ q) ⇒ (¬q ⇒ ¬p)` (on l'a fait)
- `⊢ (p ∨ q) ⇒ ((p ⇒ r) ⇒ ((q ⇒ r) ⇒ r))` (disjonction complète)
- `⊢ (p ⇒ (q ⇒ r)) ⇒ ((p ∧ q) ⇒ r)` (dé-curryfication)

Tu peux les écrire **tous** dans le même tableau à 3 colonnes comme on a fait.

Mini-guide pour l'élève (texte que tu peux coller)

Pour faire une preuve comme dans Edukera :

1. Regarder si la formule à prouver commence par ⇒. Si oui, on écrit "montrons …" puis on ouvre avec "supposons h : …".
2. À l'intérieur, on utilise seulement : ⇒E, ∧E, ∧I, ∨I, parfois ¬I.
3. Si on doit prouver une disjonction à partir de `p ∨ q`, on fait 2 sous-preuves (cas p / cas q) puis on ferme avec ∨E.
4. Si on doit prouver une négation, on suppose l'affirmation et on fabrique ⊥.
5. Quand on ferme une sous-preuve, on indique toujours les numéros : "par ⇒I sur (2)–(5)".
6. On garde le même alignement et le même vocabulaire : **montrons**, **supposons**, **par ⇒E**, **par ∧I**, **par ¬I**.

Conclusion générale – "quoi faire dans chaque cas" (style Edukera)

Quand tu dois faire une preuve en déduction naturelle comme on vient de faire, tu ne réfléchis pas 1000 ans. Tu regardes juste **la tête de la formule à prouver** et tu appliques la recette qui va avec. Voilà toutes les recettes.

1. Si la conclusion est une implication `A ⇒ B`

➤ Tu écris : `montrons A ⇒ B`
➤ Tu ouvres une boîte : `supposons h : A`
➤ Dans cette boîte, ton **objectif local** devient : prouver `B`
➤ Tu utilises les hypothèses déjà là (celles du début + `h`) avec les règles autorisées (⇒E, ∧E, …) pour obtenir `B`
➤ Quand tu as `B`, tu fermes : `A ⇒ B` par ⇒I sur les lignes de la boîte.

Donc : "conclusion commence par ⇒" → "je suppose la partie de gauche". Toujours.

2. Si la conclusion est une négation `¬A`

➤ Tu écris : `montrons ¬A`
➤ Tu ouvres une boîte : `supposons h : A`
➤ Tu cherches à fabriquer `⊥` (une contradiction) en combinant `h` avec ce que tu as (`¬A`, ou un

truc qui donne le contraire, ou un OU qui donne une absurdité…)
➤ Dès que tu as $\bot$ dans la boîte, tu peux fermer : $\neg A$ par ¬I sur les lignes de la boîte.

Donc : “conclusion est une négation” → “je suppose ce qu’on nie et je fabrique ⊥”.

3. Si la conclusion est une conjonction $A \wedge B$

➤ Il faut avoir A et B séparément
➤ Dès que tu as A quelque part (ligne 4) et B quelque part (ligne 7) → tu écris : $A \wedge B$ par ∧I avec (4), (7).
➤ Si tu ne les as pas, tu vas les chercher avec d’autres règles (⇒E, ∨E, …)

Donc : “conclusion est un ∧” → “je prouve les deux morceaux puis ∧I”.

4. Si la conclusion est une disjonction $A \vee B$

➤ Si tu as déjà A, tu peux écrire $A \vee B$ par $\vee I_1$
➤ Si tu as déjà B, tu peux écrire $A \vee B$ par $\vee I_2$
➤ Si tu n’as ni A ni B → ce n’est **pas** une bonne conclusion pour ce moment-là, il faut d’abord fabriquer A ou B.

Donc : “conclusion est un ∨” → “je mets dedans ce que j’ai déjà”.

5. Si tu as dans les hypothèses un OU $P \vee Q$ et qu’on te demande de montrer R

➤ C’est le cas typique de **∨E**
➤ Tu fais 2 sous-preuves :

- cas 1 : `supposons P` … tu montres R
- cas 2 : `supposons Q` … tu montres R

➤ Puis tu écris : R par ∨E avec (ligne du $P \vee Q$), (bloc P→R), (bloc Q→R).

Donc : “j’ai un OU dans les hypothèses et une conclusion commune” → “je fais 2 cas et je ferme avec ∨E”.

6. Si tu veux utiliser une implication que tu as déjà $A \Rightarrow B$

➤ Regarde si tu as A quelque part.
➤ Si oui → tu peux tout de suite écrire B par ⇒E.
➤ Si non → il faut d’abord fabriquer A (par ex. en supposant A dans une sous-preuve, ou en le déduisant d’une conjonction).

Donc : “j’ai (A⇒B)” → “je cherche A pour faire ⇒E”.

7. Si tu vois qu’une contradiction est possible

➤ tu écris juste $\bot$ en citant les deux lignes contradictoires (par ex. 5 : B, 6 : ¬B).
➤ après ⊥, tu peux :

- soit fermer une négation (par ¬I)
- soit dériver n’importe quoi (par ⊥E), par ex. pour terminer une implication

8. L'ordre des boîtes

➤ on ouvre dans l'ordre où les ⇒ apparaissent dans la conclusion
➤ on ferme dans l'ordre inverse (dernier ouvert = premier fermé)
➤ dans la justification on note toujours "k–m" pour dire "c'est la boîte ouverte ligne k et fermée ligne m".

9. Les 5 phrases à réutiliser (toujours les mêmes)

- `montrons …` (pour annoncer l'objectif)
- `supposons h : …` (pour ouvrir une hypothèse)
- `par ⇒E avec …` (pour modus ponens)
- `par ∧I avec …` (pour coller deux résultats)
- `par ⇒I sur …–…` ou `par ¬I sur …–…` (pour fermer)

10. Résumé ultra-court

• concl. = ⇒ → suppose la gauche
• concl. = ¬ → suppose l'affirmation et fais ⊥
• concl. = ∧ → prouve les deux puis ∧I
• concl. = ∨ → prends ce que tu as puis ∨I
• hyp. = P ∨ Q → fais 2 cas puis ∨E
• hyp. = A ⇒ B et tu as A → fais ⇒E
• contradiction B / ¬B → écris ⊥ puis ferme

Si tu appliques juste ça, toutes les preuves du TD4 ont la même forme et sont lisibles comme dans Edukera.

Conclusion + guide complet (version longue TD4 / Edukera)

Ce bloc est fait pour être **copié-collé** tel quel pour les étudiants. Il explique quoi faire dans *toutes* les situations classiques du TD4 (déduction naturelle, style Edukera), avec la même présentation que dans les captures.

0. Objectif du TD4

Dans ce TD on ne "réfléchit" pas comme en maths libre. On **applique un petit nombre de règles** (`⇒I`, `⇒E`, `∧I`, `∧E`, `¬I`, `∨I`, `∨E`, `⊥E`) suivant un **schéma de colonnes** : numéro de ligne – formule – règle. Le but est que **chaque ligne** dise clairement : "j'ai écrit cette formule *à cause de* cette règle, et en utilisant ces lignes".

1. Modèle général à apprendre par cœur

Quand tu ouvres ton exercice, tu commences par regarder **la forme de la conclusion**. C'est ça qui décide de la suite.

Conclusion à prouver	Action à faire	Règle à écrire

Conclusion à prouver	Action à faire	Règle à écrire
`A ⇒ B`	ouvrir une boîte avec `supposons A`, prouver B dedans, fermer	`⇒I`
`¬A`	ouvrir une boîte avec `supposons A`, fabriquer ⊥, fermer	`¬I`
`A ∧ B`	prouver A, prouver B, coller	`∧I`
`A ∨ B`	si tu as A → ∨I₁, si tu as B → ∨I₂	`∨I`
`R` et tu as `P ∨ Q`	faire 2 cas : supp P ⟶ R, supp Q ⟶ R	`∨E`
rien ne marche / tu vois un couple B et ¬B	écrire ⊥ puis fermer	`⊥E` ou `¬I`

Si tu appliques juste cette table, tu fais 90% du TD.

2. Rappel du format “Edukera” (3 colonnes, boîtes imbriquées)

On veut exactement ce style (comme dans ta photo) :

#	Formule	Règle
1	…	hyp.
2	…	…

Quand on est dans une sous-preuve, on décale la formule (petit margin-left dans ton HTML) ou on met “(sous-preuve)”. À la fermeture de la sous-preuve, on revient au niveau précédent.

3. Exemple de base à bien comprendre (déjà vu) – Loi du tiers exclu

Objectif : montrer `A ∨ ¬A` (exemple classique que tu as déjà écrit)

#	Formule	Règle
(1)	**montrons** `A ∨ ¬A`	
(2)	**supposons** `h₁ : ¬(A ∨ ¬A)`	hyp. pour ¬I
(3)	**montrons** `¬A`	objectif local
(4)	**supposons** `h₂ : A`	hyp.
(5)	`A ∨ ¬A`	∨I₁ avec (4)
(6)	`⊥`	E¬ avec (2), (5)
(7)	`¬A`	¬I 4–6
(8)	`A ∨ ¬A`	∨I₂ avec (7)
(9)	`⊥`	E¬ avec (2), (8)
(10)	**A ∨ ¬A**	¬I 2–9

➤ ce modèle te montre : “je veux une disjonction → j’en fabrique une dans une sous-preuve, puis je reviens et je ferme”.

4. Exemple “prendre une conjonction et en sortir un bout”

But : on me donne `p ∧ q`. Je veux montrer `p`.

#	Formule	Règle
(1)	p ∧ q	hyp.
(2)	p	∧E₁ sur (1)

➤ très court, mais c’est CE que tu dois écrire ; on nomme la règle.

5. Exemple : “j’ai un ∧ et un ⇒, je veux la conclusion”

Énoncé type : de `p ∧ (p ⇒ q)`, déduire `q`.

#	Formule	Règle
(1)	p ∧ (p ⇒ q)	hyp.
(2)	p	∧E₁ sur (1)
(3)	p ⇒ q	∧E₂ sur (1)
(4)	q	⇒E avec (3), (2)

➤ c’est un schéma super fréquent : “je décroche des trucs avec ∧E, puis j’utilise ⇒E”.

6. Exemple : “implication dans l’implication”

But : montrer `(p ∧ q) ⇒ p`.

#	Formule	Règle
(1)	**montrons** (p ∧ q) ⇒ p	
(2)	**supposons** h : p ∧ q	hyp. (pour ⇒I)
(3)	p	∧E₁ sur (2)
(4)	(p ∧ q) ⇒ p	⇒I 2–3

➤ c’est LE modèle “montrer que de p∧q on peut sortir p”. Tu peux faire le même avec q.

7. Exemple : preuve par cas (avec ∨E) – modèle à répéter

Énoncé type : On sait `P ∨ Q`, on sait `P ⇒ R`, on sait `Q ⇒ R`, montrer `R`.

#	Formule	Règle
(1)	P ∨ Q	hyp.
(2)	P ⇒ R	hyp.

#	Formule	Règle
(3)	Q ⇒ R	hyp.

```
 <!-- cas P -->
  <tr>
    <td style="padding-left:20px;">(4)</td>
    <td><strong>supposons</strong> P</td>
    <td>hyp. (cas 1)</td>
  </tr>
  <tr>
    <td style="padding-left:20px;">(5)</td>
    <td>R</td>
    <td>⇒E avec (2), (4)</td>
  </tr>

  <!-- cas Q -->
  <tr>
    <td style="padding-left:20px;">(6)</td>
    <td><strong>supposons</strong> Q</td>
    <td>hyp. (cas 2)</td>
  </tr>
  <tr>
    <td style="padding-left:20px;">(7)</td>
    <td>R</td>
    <td>⇒E avec (3), (6)</td>
  </tr>

  <!-- sortie -->
  <tr><td>(8)</td><td><strong>R</strong></td><td>∨E avec (1), 4–5, 6–
7</td></tr>
</tbody>
```

➤ exactement le même dessin que dans ta photo, mais en version complète. ➤ à réutiliser pour tous les exos “X, Y, Z ont dit …”.

8. Exemple : contraposée constructive (important)

But : montrer `(A ⇒ B) ⇒ (¬B ⇒ ¬A)` sans utiliser la contraposée directe.

#	Formule	Règle
(1)	**supposons** A ⇒ B	hyp.
(2)	**supposons** ¬B	hyp.
(3)	**supposons** A	hyp. (pour faire ¬A)
(4)	B	⇒E avec (1), (3)
(5)	⊥	E¬ avec (2), (4)
(6)	¬A	¬I 3–5

#	Formule	Règle
(7)	¬B ⇒ ¬A	⇒I 2–6
(8)	(A ⇒ B) ⇒ (¬B ⇒ ¬A)	⇒I 1–7

➤ ça, c'est un exercice que les profs aiment. Si tu sais écrire exactement ça, tu as le style demandé.

9. Ce qu'il faut faire quand on est bloqué

- **Regarde la conclusion** : est-ce une implication ? une négation ? un ∧ ? un ∨ ?
- **S'il y a un ⇒** → OUVRE une hypothèse.
- **S'il y a un ¬** → OUVRE une hypothèse sur ce qu'on nie.
- **S'il y a un ∧** → cherche les 2 morceaux séparés.
- **Tu as une formule du type A ⇒ B dans les hypothèses ?** → cherche A pour faire ⇒E.
- **Tu vois B et ¬B dans la même boîte ?** → écris ⊥ puis ferme.

10. Erreurs à NE PAS faire (à dire aux étudiants)

- ne jamais écrire une formule sans écrire la règle à droite ;
- ne pas utiliser une règle qui n'est pas dans la liste du TD ;
- ne pas sauter une fermeture de boîte (si tu ouvres 3 boîtes tu en fermes 3) ;
- ne pas mélanger les niveaux (une ligne dans une sous-preuve ne peut pas utiliser une ligne qui est ouverte après) ;
- ne pas oublier de citer *toutes* les lignes dans ∨E (il faut ligne du ∨ + bloc 1 + bloc 2).

11. Mini-banque d'exercices à faire pareil

(a) De p ⇒ (q ⇒ r), déduire (p ∧ q) ⇒ r

Idée : on veut une implication, donc on suppose p ∧ q ; on décroche p et q ; on utilise p ⇒ (q ⇒ r) ; on applique deux fois ⇒E.

#	Formule	Règle
(1)	p ⇒ (q ⇒ r)	hyp.
(2)	**supposons** p ∧ q	hyp. (pour ⇒I)
(3)	p	∧E₁ sur (2)
(4)	q	∧E₂ sur (2)
(5)	q ⇒ r	⇒E avec (1), (3)
(6)	r	⇒E avec (5), (4)
(7)	(p ∧ q) ⇒ r	⇒I 2–6

(b) De (p ⇒ q) et (p ⇒ r), déduire p ⇒ (q ∧ r)

#	Formule	Règle
(1)	p ⇒ q	hyp.

#	Formule	Règle
(2)	p ⇒ r	hyp.
(3)	**supposons** p	hyp. (pour ⇒I)
(4)	q	⇒E avec (1), (3)
(5)	r	⇒E avec (2), (3)
(6)	q ∧ r	∧I avec (4), (5)
(7)	p ⇒ (q ∧ r)	⇒I 3–6

(c) De ¬p ⇒ ⊥, déduire p (si la règle ⊥E est autorisée)

#	Formule	Règle
(1)	¬p ⇒ ⊥	hyp.
(2)	**supposons** ¬p	hyp.
(3)	⊥	⇒E avec (1), (2)
(4)	p	⊥E sur (3)
(5)	¬p ⇒ p	⇒I 2–4

➤ à adapter selon les règles autorisées par ton enseignant.

12. Vraie conclusion (à donner aux étudiants)

Pour réussir ce TD et avoir des preuves “propres” :

1. **Regarder la forme de la conclusion.**
2. **Ouvrir les hypothèses nécessaires** (une par ⇒, une par négation, deux pour ∨E).
3. **À l’intérieur**, n’utiliser que : ⇒E, ∧E, ∧I, ∨I, parfois ⊥E.
4. **Fermer** dans l’ordre inverse avec ⇒I ou ¬I en citant les lignes de début et de fin.
5. **Nommer toutes les règles** exactement comme dans le cours.

Si tu fais ça, **toutes** tes preuves auront le même dessin que celles d’Edukera, et le correcteur verra tout de suite ce que tu fais.

TD4 – Déduction naturelle (style Edukera)

> Dans ce fichier tu as **tout ce qu’il faut** pour faire les preuves comme sur la capture : mêmes couleurs, mêmes boîtes, mêmes “supposons…”, mêmes “montrons…”. L’idée : tu donnes un **cours très court** au début, puis tu enchaînes sur les **modèles**.

---

1. Rappel très court : ce qu’on fait en TD4

En TD4 on ne calcule pas une valeur de vérité. On **construit une preuve** ligne par ligne avec les **règles de déduction naturelle** :

- `⇒I` introduction de l'implication
- `⇒E` (modus ponens)
- `∧I` / `∧E`
- `∨I₁`, `∨I₂`, `∨E`
- `¬I` (faire une contradiction pour obtenir une négation)
- `⊥E` (de ⊥ on peut tout déduire)

On doit **toujours** écrire la règle à droite + les lignes utilisées. On écrit aussi **montrons…** et **supposons…** comme dans le PDF.

---

2. Règle de décision par forme de conclusion

Conclusion à prouver	Ce que tu fais	Règle
`A ⇒ B`	Ouvrir une boîte : `supposons A`, prouver B dedans, fermer.	`⇒I`
`¬A`	Ouvrir une boîte : `supposons A`, faire ⊥, fermer.	`¬I`
`A ∧ B`	Prouver A, prouver B, coller.	`∧I`
`A ∨ B`	Prouver A → `∨I₁` ou prouver B → `∨I₂`.	`∨I`
Tu as `P ∨ Q` en hyp. et tu veux `R`	Faire 2 cas : `supposons P …` puis `supposons Q …`, fermer.	`∨E`
Tu vois `B` et `¬B`	Écrire ⊥ puis fermer / exploser.	`⊥E` ou `¬I`

---

3. Modèles simples

3.1 Modèle "j'ai les deux, je colle"

(1)	p	hyp.
(2)	q	hyp.
(3)	**p ∧ q**	∧I avec (1), (2)

3.2 Modèle "modus ponens"

(1)	p ⇒ q	hyp.
(2)	p	hyp.
(3)	**q**	⇒E avec (1), (2)

3.3 Modèle “conclusion commence par ⇒”

But : ⊢ (p ∧ q) ⇒ p

(1)	**montrons** (p ∧ q) ⇒ p	
(2)	**supposons** h : p ∧ q	hyp. (pour ⇒I)
(3)	p	∧E₁ sur (2)
(4)	**(p ∧ q) ⇒ p**	⇒I 2–3

---

4. Exemple “contraposée constructive”

But : ⊢ (A ⇒ B) ⇒ (¬B ⇒ ¬A)

(1)	**montrons** (A ⇒ B) ⇒ (¬B ⇒ ¬A)	
(2)	**supposons** h_1 : A ⇒ B	hyp.
(3)	**supposons** h_2 : ¬B	hyp.
(4)	**supposons** h_3 : A	hyp. (pour faire ¬A)
(5)	B	⇒E avec (2), (4)
(6)	⊥	E¬ avec (3), (5)
(7)	**¬A**	¬I 4–6
(8)	**¬B ⇒ ¬A**	⇒I 3–7
(9)	**(A ⇒ B) ⇒ (¬B ⇒ ¬A)**	⇒I 2–8

---

5. Exemple “tiers exclu” (comme ta diapo)

But : ⊢ A ∨ ¬A

(1)	**montrons** A ∨ ¬A	
(2)	**supposons** h_1 : ¬(A ∨ ¬A)	hyp. pour ¬I
(3)	**montrons** ¬A	objectif local
(4)	**supposons** h_2 : A	hyp.
(5)	A ∨ ¬A	∨I₁ avec (4)
(6)	⊥	E¬ avec (2), (5)
(7)	**¬A**	¬I 4–6
(8)	A ∨ ¬A	∨I₂ avec (7)
(9)	⊥	E¬ avec (2), (8)
(10)	**A ∨ ¬A**	¬I 2–9

---

6. Dernier exo “un peu sale” (celui de ton screenshot)

À prouver :

[((P ⇒ Q) ∧ (R ⇒ S) ∧ (P ∨ R)) ⇒ (Q ∨ S)]

Idée : on sort d’abord les trois morceaux, ensuite on fait **deux cas** (P / R), dans chaque cas on fabrique **Q ∨ S**, puis on ferme avec **∨E**, puis on ferme l’implication.

(1)	**supposons** H : (P ⇒ Q) ∧ (R ⇒ S) ∧ (P ∨ R)	hyp. pour ⇒I
(2)	(P ⇒ Q) ∧ (R ⇒ S)	∧E₁ sur (1)
(3)	P ∨ R	∧E₂ sur (1)
(4)	P ⇒ Q	∧E₁ sur (2)
(5)	R ⇒ S	∧E₂ sur (2)
(6)	**supposons** P	hyp. (cas 1 pour ∨E)
(7)	Q	⇒E avec (4), (6)
(8)	Q ∨ S	∨I₁ avec (7)
(9)	**supposons** R	hyp. (cas 2 pour ∨E)
(10)	S	⇒E avec (5), (9)
(11)	Q ∨ S	∨I₂ avec (10)
(12)	**Q ∨ S**	∨E avec (3), (6–8), (9–11)
(13)	**((P ⇒ Q) ∧ (R ⇒ S) ∧ (P ∨ R)) ⇒ (Q ∨ S)**	⇒I 1–12

---

7. Comment l’enseigner (texte à dire / mettre dessous)

Si je veux réussir tous les exos de ce type :

1. Je regarde la tête de la conclusion.
2. S’il y a un ⇒ je commence par “supposons …”.
3. Dedans je n’utilise que 4 trucs : ∧E pour décrocher, ⇒E pour avancer, ∧I pour coller, ∨I/∨E pour les OU.
4. Si j’ai un OU en hypothèse, je fais 2 cas complets (cas 1 … cas 2 …) puis je ferme.
5. Quand je ferme, j’écris toujours “⇒I 2–5” ou “¬I 3–7”.
6. Je n’utilise pas de règle que le prof n’a pas autorisée.

Le dernier exercice juste au-dessus est un très bon “stress test” : tu ouvres, tu décroches, tu fais 2 cas, tu reconstruis un ∨ en sortie. Si tu sais faire ça, tu peux faire presque tous les exos du cours.

Exercice 14 – Preuves de formules
On doit prouver les formules suivantes en utilisant **uniquement** les règles de déduction naturelle (∧I, ∧E, ∨I₁, ∨I₂, ∨E, ⇒I, ⇒E, ¬I, E¬, ⊥E).
Interdit : raisonnement par l’absurde global, tiers exclu, règles dérivées.

1. F1 = (A ⇒ B) ⇒ ((B ⇒ C) ⇒ (A ⇒ C))
2. F2 = (((A ∧ B) ⇒ C) ∧ (A ⇒ B)) ⇒ (A ⇒ C)
3. F3 = (A ⇒ B) ⇒ ((A ∨ C) ⇒ (B ∨ C))
4. F4 = (A ∨ (B ⇒ C)) ⇒ ((A ∨ B) ⇒ (A ∨ C))
5. F5 = (A ⇒ B) ⇒ (¬B ⇒ ¬A)
6. F6 = (A ∧ B) ⇒ ¬(¬A ∨ ¬B)

Exercice 14 – Question 1
Prouver : `F1 = (A ⇒ B) ⇒ ((B ⇒ C) ⇒ (A ⇒ C))`

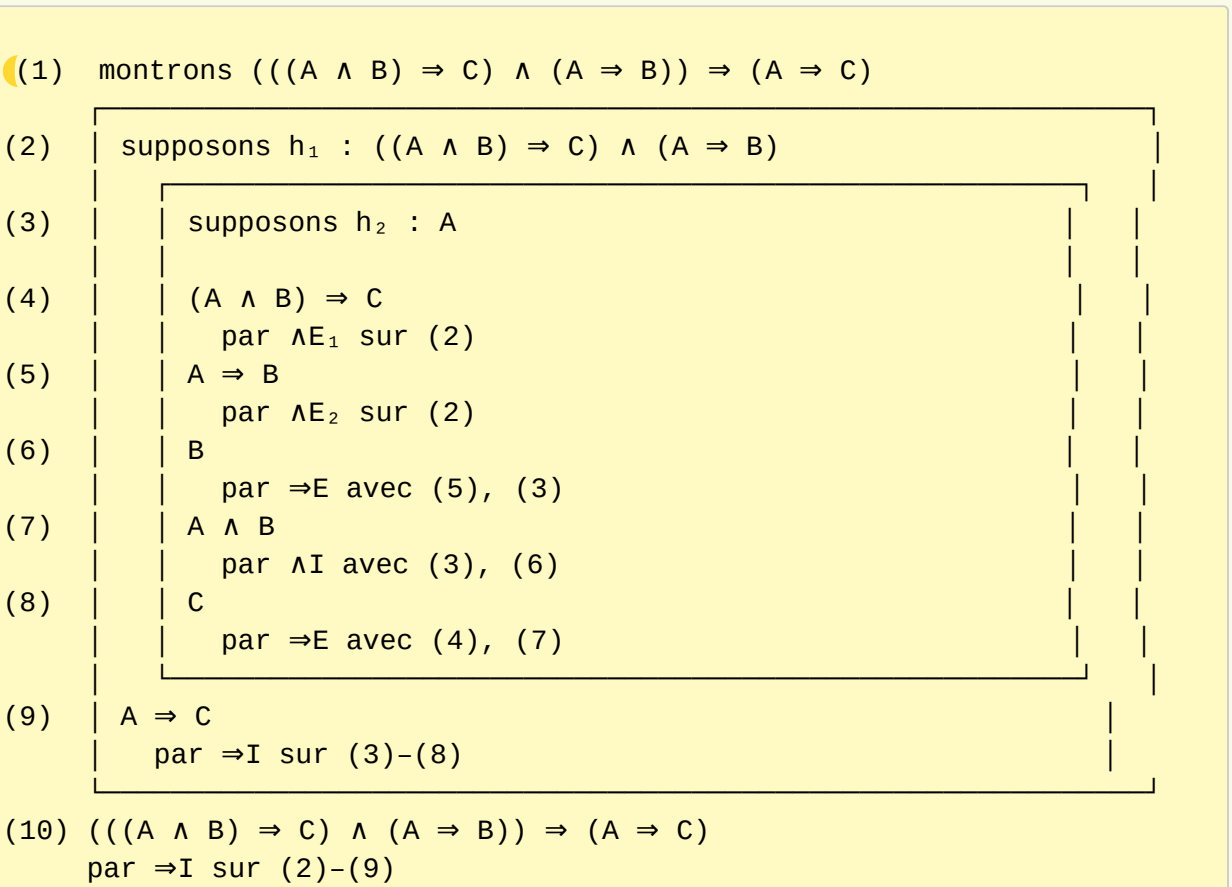
```
(1)  montrons (A ⇒ B) ⇒ ((B ⇒ C) ⇒ (A ⇒ C))
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : A ⇒ B                             │
     │   ┌──────────────────────────────────────────┐   │
(3)  │   │ supposons h₂ : B ⇒ C                     │   │
     │   │   ┌──────────────────────────────────┐   │   │
(4)  │   │   │ supposons h₃ : A                 │   │   │
(5)  │   │   │ B                                │   │   │
     │   │   │   par ⇒E avec (2), (4)           │   │   │
(6)  │   │   │ C                                │   │   │
     │   │   │   par ⇒E avec (3), (5)           │   │   │
     │   │   └──────────────────────────────────┘   │   │
(7)  │   │ A ⇒ C                                    │   │
     │   │   par ⇒I sur (4)–(6)                     │   │
     │   └──────────────────────────────────────────┘   │
(8)  │ (B ⇒ C) ⇒ (A ⇒ C)                                │
     │   par ⇒I sur (3)–(7)                             │
     └──────────────────────────────────────────────────┘
(9)  (A ⇒ B) ⇒ ((B ⇒ C) ⇒ (A ⇒ C))
     par ⇒I sur (2)–(8)
```

Idée. La conclusion est une grosse implication avec 3 “⇒”. On les déroule **un par un** :

- on suppose d’abord `A ⇒ B`,
- puis on suppose `B ⇒ C`,
- puis, pour faire `A ⇒ C`, on suppose `A` et on fabrique `C`.

On utilise seulement **⇒E** (modus ponens) et **⇒I** (fermeture de la boîte).

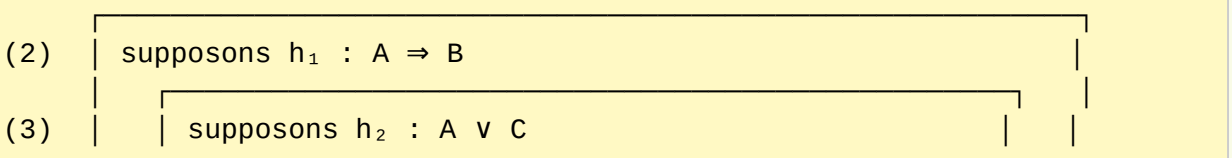
Exercice 14 – Question 2
Prouver : `F2 = (((A ∧ B) ⇒ C) ∧ (A ⇒ B)) ⇒ (A ⇒ C)`



Comment on a pensé ?
La conclusion est `(A ⇒ C)`, donc on commence par `supposons A`. Pour faire `C` on a un `(A ∧ B) ⇒ C`, donc on doit d'abord construire `A ∧ B`. On a déjà `A` (hypothèse locale) et on obtient `B` en appliquant l'implication `A ⇒ B` qu'on a sortie du gros ∧. Une fois qu'on a `A ∧ B`, on applique l'implication vers `C`.

Règles utilisées : ∧E₁, ∧E₂, ⇒E, ∧I, ⇒I. Aucune règle interdite.

Exercice 14 – Question 3
Prouver : `F3 = (A ⇒ B) ⇒ ((A ∨ C) ⇒ (B ∨ C))`



```
      |   |                                                   |   |
      |   |   ┌──────────── cas A ────────────┐               |   |
(4)   |   |   | supposons h₃ : A              |               |   |
(5)   |   |   | B                             |               |   |
      |   |   |   par ⇒E avec (2), (4)        |               |   |
(6)   |   |   | B ∨ C                         |               |   |
      |   |   |   par ∨I₁ avec (5)            |               |   |
      |   |   └───────────────────────────────┘               |   |
      |   |   ┌──────────── cas C ────────────┐               |   |
(7)   |   |   | supposons h₄ : C              |               |   |
(8)   |   |   | B ∨ C                         |               |   |
      |   |   |   par ∨I₂ avec (7)            |               |   |
      |   |   └───────────────────────────────┘               |   |
(9)   |   | B ∨ C                                             |   |
      |   |   par ∨E avec (3), (4)-(6), (7)-(8)               |   |
      |   └───────────────────────────────────────────────────┘   |
(10)  | (A ∨ C) ⇒ (B ∨ C)                                         |
      |   par ⇒I sur (3)-(9)                                      |
      └───────────────────────────────────────────────────────────┘
(11) (A ⇒ B) ⇒ ((A ∨ C) ⇒ (B ∨ C))
     par ⇒I sur (2)-(10)
```

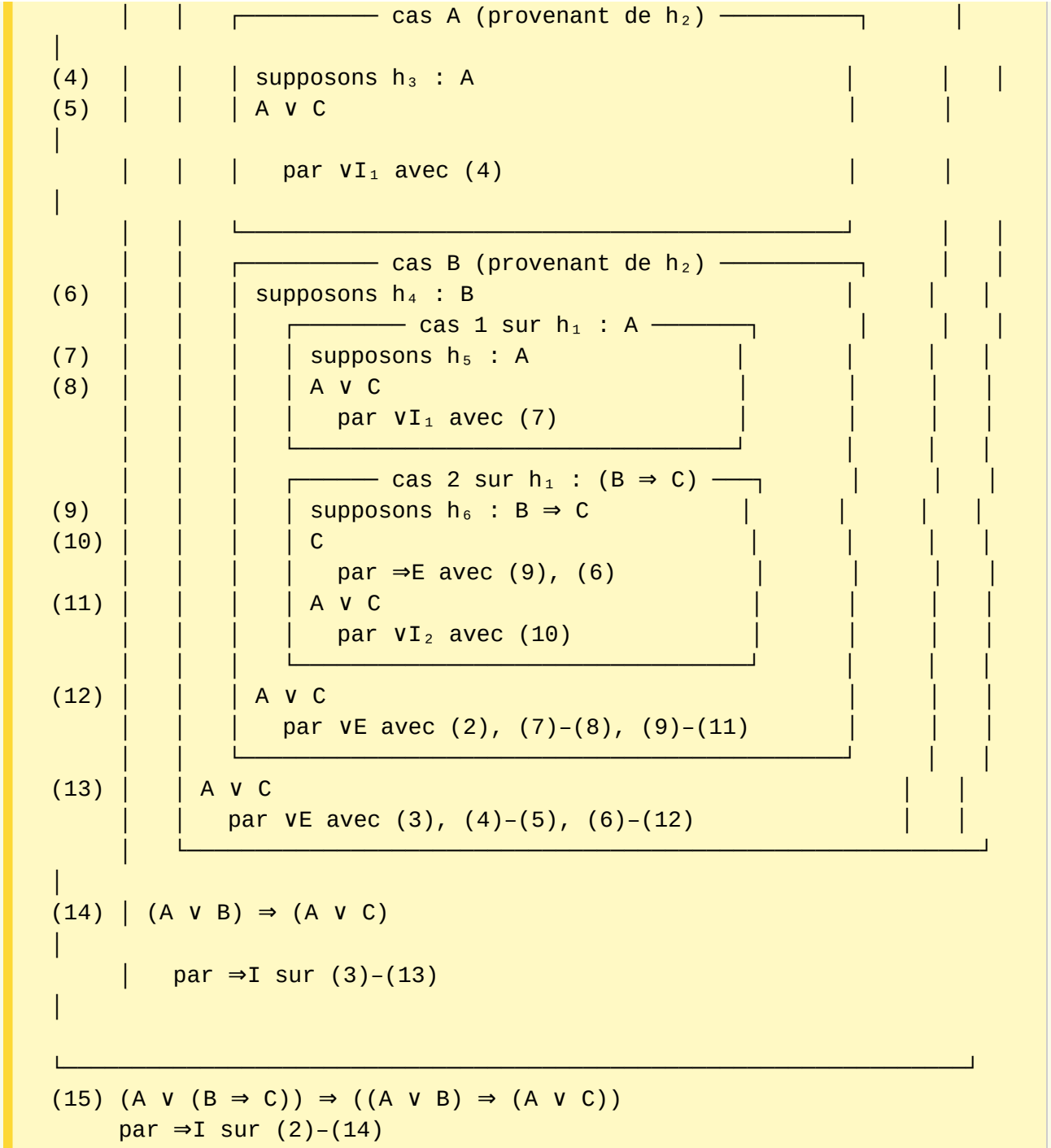
Idée. On veut sortir d'un `A ∨ C`, donc on est obligé de faire une `∨E` avec deux sous-preuves : cas `A` et cas `C`. Dans le cas `A`, on transforme `A` en `B` grâce à l'implication de départ, puis on met dans un OU. Dans le cas `C`, on met directement dans le OU à droite.

C'est exactement le schéma "j'ai un OU dans les hypothèses → je fais 2 cas → je ferme avec ∨E".

Exercice 14 – Question 4
Prouver : `F4 = (A ∨ (B ⇒ C)) ⇒ ((A ∨ B) ⇒ (A ∨ C))`

```
(1)  montrons (A ∨ (B ⇒ C)) ⇒ ((A ∨ B) ⇒ (A ∨ C))

┌──────────────────────────────────────────────────────────────┐
(2)  | supposons h₁ : A ∨ (B ⇒ C)
|
     |   ┌──────────────────────────────────────────────────┐
|
(3)  |   | supposons h₂ : A ∨ B                             |
|
     |   |                                                  |
|
```



Pourquoi c’est plus long ?
Parce qu’on a un `∨` dans l’hypothèse extérieure **et** un `∨` dans l’hypothèse intérieure. On doit donc faire **2 niveaux de ∨E**.

- On veut : à partir de `A ∨ B` → `A ∨ C`. - Si c’est `A` : fini. - Si c’est `B` : on regarde l’autre disjonction `A ∨ (B ⇒ C)`. * si c’est `A` : fini * si c’est `B ⇒ C` : on applique sur le `B` qu’on a, on obtient `C`, puis on met dans `A ∨ C`.

C’est exactement la même mécanique que tes screenshots Edukera : on ouvre, on fait cas interne, on re-ferme.

Exercice 14 – Question 5
Prouver : `F5 = (A ⇒ B) ⇒ (¬B ⇒ ¬A)`

```
(1)  montrons (A ⇒ B) ⇒ (¬B ⇒ ¬A)
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : A ⇒ B                             │
     │   ┌──────────────────────────────────────────┐   │
(3)  │   │ supposons h₂ : ¬B                        │   │
     │   │   ┌──────────────────────────────────┐   │   │
(4)  │   │   │ supposons h₃ : A                 │   │   │
(5)  │   │   │ B                                │   │   │
     │   │   │   par ⇒E avec (2), (4)           │   │   │
(6)  │   │   │ ⊥                                │   │   │
     │   │   │   par E¬ avec (3), (5)           │   │   │
     │   │   └──────────────────────────────────┘   │   │
(7)  │   │ ¬A                                       │   │
     │   │   par ¬I sur (4)–(6)                     │   │
     │   └──────────────────────────────────────────┘   │
(8)  │ ¬B ⇒ ¬A                                          │
     │   par ⇒I sur (3)–(7)                             │
     └──────────────────────────────────────────────────┘
(9)  (A ⇒ B) ⇒ (¬B ⇒ ¬A)
     par ⇒I sur (2)–(8)
```

C'est la **contraposée constructive** faite "à la main" : pour montrer ¬A on suppose A, on obtient B, on croise avec ¬B, on fait ⊥, puis on ferme.

On n'a jamais utilisé de "si non-B alors non-A" comme règle automatique : on la prouve.

Exercice 14 – Question 6
Prouver : `F6 = (A ∧ B) ⇒ ¬(¬A ∨ ¬B)`

```
(1)  montrons (A ∧ B) ⇒ ¬(¬A ∨ ¬B)
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : A ∧ B                             │
     │   ┌──────────────────────────────────────────┐   │
(3)  │   │ supposons h₂ : ¬A ∨ ¬B                   │   │
     │   │                                          │   │
(4)  │   │ A                                        │   │
```

```
      |   |   par ∧E₁ sur (2)                                          |   |
(5)   |   | B                                                          |   |
      |   |   par ∧E₂ sur (2)                                          |   |
      |   |   ┌──────────── cas ¬A ────────────┐                       |   |
(6)   |   |   | supposons h₃ : ¬A              |                     |   |
(7)   |   |   | ⊥                              |                     |   |
      |   |   |   par E¬ avec (6), (4)         |                     |   |
      |   |   └────────────────────────────────┘                       |   |
      |   |   ┌──────────── cas ¬B ────────────┐                       |   |
(8)   |   |   | supposons h₄ : ¬B              |                     |   |
(9)   |   |   | ⊥                              |                     |   |
      |   |   |   par E¬ avec (8), (5)         |                     |   |
      |   |   └────────────────────────────────┘                       |   |
(10)  |   | ⊥                                                          |   |
      |   |   par ∨E avec (3), (6)–(7), (8)–(9)                        |   |
      |   └────────────────────────────────────────────────────────────┘   |
(11)  | ¬(¬A ∨ ¬B)                                                         |
      |   par ¬I sur (3)–(10)                                              |
      └────────────────────────────────────────────────────────────────────┘

(12) (A ∧ B) ⇒ ¬(¬A ∨ ¬B)
     par ⇒I sur (2)–(11)
```

Pour montrer une négation, on suppose ce qu'on nie (`¬A ∨ ¬B`) et on fabrique ⊥. Ici ⊥ vient du fait qu'on possède aussi `A` et `B` (décrochés du ∧).

Comme c'est un `∨` qu'on nie, on est obligé de faire **2 cas** (¬A / ¬B) puis de fermer avec `∨E`.

Exercice 15 – Enigme
On a 3 affirmations sur Anissa, Boris et Chloé. On veut montrer que Chloé ne vient pas.

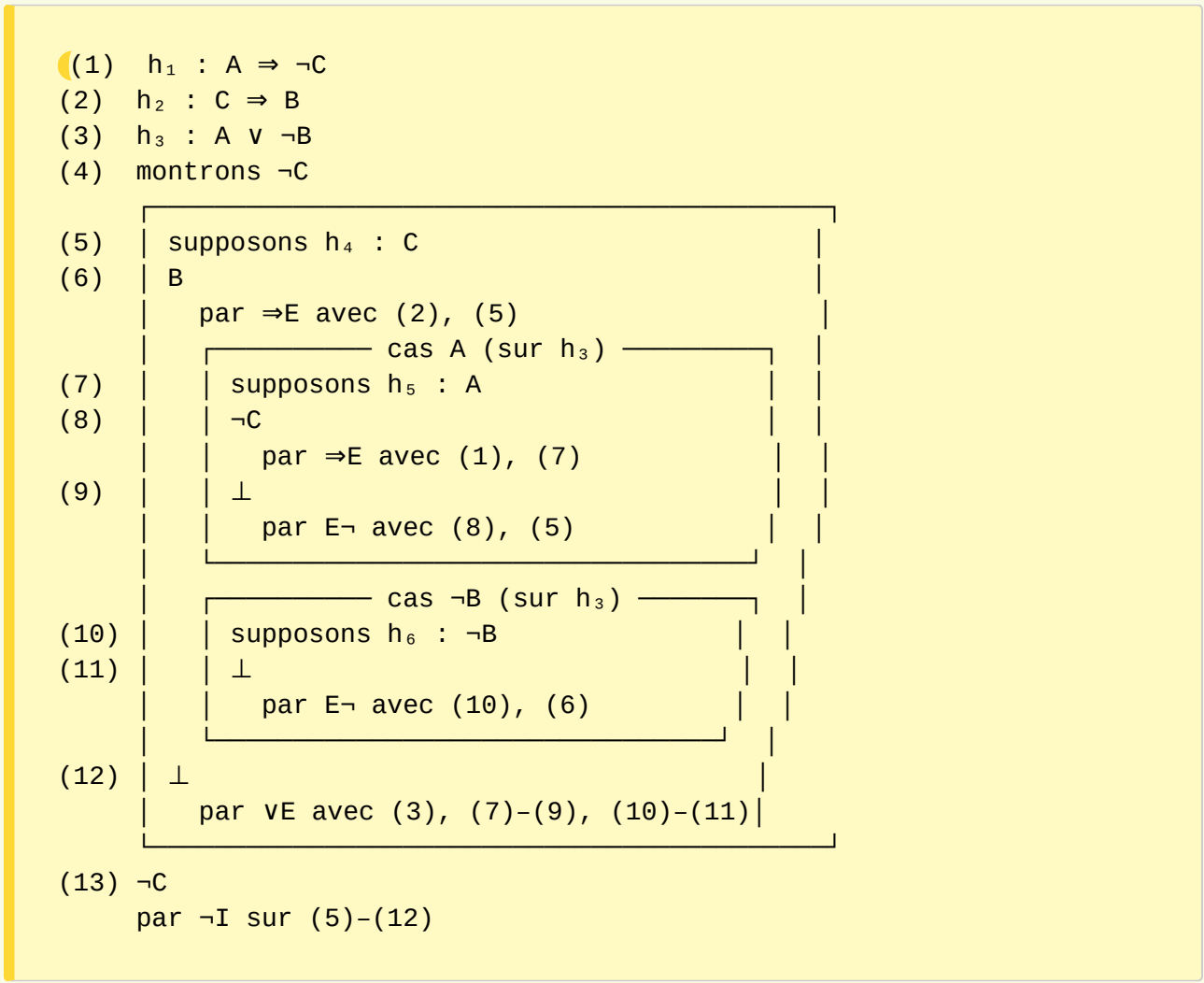
Exercice 15 – Question 1 : formalisation
On dispose des propositions atomiques :
• A : "Anissa vient"
• B : "Boris vient"
• C : "Chloé vient"
Formaliser :

- h1 : Si Anissa vient, alors Chloé ne vient pas.
- h2 : Si Chloé vient, alors Boris vient.
- h3 : Anissa vient ou Boris ne vient pas.

h1 : $A \Rightarrow \neg C$
h2 : $C \Rightarrow B$
h3 : $A \vee \neg B$

Rien de spécial ici : “ne vient pas” → négation, “ou” → ∨, “si … alors …” → ⇒.

Exercice 15 – Question 2 : preuve
Montrer : à partir de h1, h2, h3, on peut déduire $\neg C$.



Méthode. On veut une négation → on suppose C. Avec C et h_2 on obtient B. Comme on a $A \vee \neg B$, on fait 2 cas :

- cas A : avec A et h_1 on obtient $\neg C$, qui contredit C → $\bot$

- cas ¬B : ça contredit directement le `B` trouvé

Comme les 2 cas donnent ⊥, la disjonction donne ⊥, donc on ferme en `¬C`.

Thème 5 – TD

Objectifs :
– Déduction naturelle, règles dérivées
– Logique classique (on a le droit au tiers exclu, au RAA)
– On peut réutiliser les schémas du Thème 4

Exercice 16 – Logique intuitionniste / logique classique
Question 1. Prouver la formule :
`F = (P ∨ Q) ⇒ ((P ⇒ Q) ∨ (Q ⇒ P))`
en n'utilisant que des règles de la logique intuitionniste.

```
(1)  montrons (P ∨ Q) ⇒ ((P ⇒ Q) ∨ (Q ⇒ P))
     ┌──────────────────────────────────────────────────────┐
(2)  │ supposons h₁ : P ∨ Q                                 │
     │                                                      │
     │   ┌──────────── cas P ────────────┐                  │
(3)  │   │ supposons h₂ : P              │                  │
(4)  │   │ supposons h₃ : Q              │                  │
(5)  │   │ P                             │                  │
     │   │   reprise de h₂               │                  │
(6)  │   │ Q ⇒ P                         │                  │
     │   │   par ⇒I sur (4)-(5)          │                  │
(7)  │   │ (P ⇒ Q) ∨ (Q ⇒ P)             │                  │
     │   │   par ∨I₂ avec (6)            │                  │
     │   └───────────────────────────────┘                  │
     │   ┌──────────── cas Q ────────────┐                  │
(8)  │   │ supposons h₄ : Q              │                  │
(9)  │   │ supposons h₅ : P              │                  │
(10) │   │ Q                             │                  │
     │   │   reprise de h₄               │                  │
(11) │   │ P ⇒ Q                         │                  │
     │   │   par ⇒I sur (9)-(10)         │                  │
(12) │   │ (P ⇒ Q) ∨ (Q ⇒ P)             │                  │
     │   │   par ∨I₁ avec (11)           │                  │
     │   └───────────────────────────────┘                  │
(13) │ (P ⇒ Q) ∨ (Q ⇒ P)                                    │
     │   par ∨E avec (2), (3)-(7), (8)-(12)                 │
     └──────────────────────────────────────────────────────┘
(14) (P ∨ Q) ⇒ ((P ⇒ Q) ∨ (Q ⇒ P))
     par ⇒I sur (2)-(13)
```

Pourquoi c'est intuitionniste ?
On n'a utilisé aucun tiers exclu, aucun RAA global. On a seulement fait :

- une **élimination du ∨** (2 cas) sur `P ∨ Q` ;
- dans le cas `P`, on a construit `Q ⇒ P` (facile car on avait déjà `P`) ;
- dans le cas `Q`, on a construit `P ⇒ Q` ;
- on a remonté avec ∨E.

Donc cette formule est bien prouvable en logique intuitionniste.

Exercice 16 – Question 2
Prouver la formule `(P ⇒ Q) ∨ (Q ⇒ P)`.
Peut-on la prouver en n'utilisant que des règles de la logique intuitionniste ?

```
(1)  montrons (P ⇒ Q) ∨ (Q ⇒ P)
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : P                                 │
(3)  │ supposons h₂ : Q                                 │
(4)  │ P                                                │
     │   reprise de h₁                                  │
(5)  │ Q ⇒ P                                            │
     │   par ⇒I sur (3)–(4)                             │
(6)  │ (P ⇒ Q) ∨ (Q ⇒ P)                                │
     │   par ∨I₂ avec (5)                               │
     └──────────────────────────────────────────────────┘

     ┌──────────────────────────────────────────────────┐
(7)  │ supposons h₃ : ¬P                                │
(8)  │ supposons h₄ : P                                 │
(9)  │ ⊥                                                │
     │   par E¬ avec (7), (8)                           │
(10) │ Q                                                │
     │   par ⊥E sur (9)                                 │
(11) │ P ⇒ Q                                            │
     │   par ⇒I sur (8)–(10)                            │
(12) │ (P ⇒ Q) ∨ (Q ⇒ P)                                │
     │   par ∨I₁ avec (11)                              │
     └──────────────────────────────────────────────────┘
(13) P ∨ ¬P                (tiers exclu, logique classique)
(14) (P ⇒ Q) ∨ (Q ⇒ P)
     par ∨E avec (13), (2)–(6), (7)–(12)
```

Réponse à la question :
– On a pu prouver `(P ⇒ Q) ∨ (Q ⇒ P)` parce qu'on a utilisé le **tiers exclu** à la ligne (13).
– **Sans** le tiers exclu (`P ∨ ¬P`), on ne peut pas choisir le bon côté de la disjonction, donc en logique intuitionniste pure, cette formule n'est pas démontrable.
– Donc : **non**, on ne peut pas prouver `(P ⇒ Q) ∨ (Q ⇒ P)` seulement avec les règles intuitionnistes.

Exercice 17 – Preuves de formules
On doit prouver les formules suivantes (logique classique autorisée) :

1. F1 = (¬B ⇒ ¬A) ⇒ (A ⇒ B)
2. F2 = ¬(A ∧ B) ⇒ (¬A ∨ ¬B)
3. F3 = ¬(A ⇒ B) ⇒ (A ∧ ¬B)
4. F4 = ¬(A ∨ ¬B) ∨ ¬(¬A ∧ B)

Exercice 17 – Question 1
Prouver : `(¬B ⇒ ¬A) ⇒ (A ⇒ B)`

```
(1)  montrons (¬B ⇒ ¬A) ⇒ (A ⇒ B)
     ┌──────────────────────────────────────────────┐
(2)  │ supposons h₁ : ¬B ⇒ ¬A                       │
     │   ┌──────────────────────────────────────┐   │
(3)  │   │ supposons h₂ : A                     │   │
     │   │   ┌──────────────────────────────┐   │   │
(4)  │   │   │ supposons h₃ : ¬B            │   │   │
(5)  │   │   │ ¬A                           │   │   │
     │   │   │   par ⇒E avec (2), (4)       │   │   │
(6)  │   │   │ ⊥                            │   │   │
     │   │   │   par E¬ avec (5), (3)       │   │   │
     │   │   └──────────────────────────────┘   │   │
(7)  │   │ B                                    │   │
     │   │   par RAA sur (4)–(6)                │   │
     │   └──────────────────────────────────────┘   │
(8)  │ A ⇒ B                                        │
     │   par ⇒I sur (3)–(7)                         │
     └──────────────────────────────────────────────┘
(9)  (¬B ⇒ ¬A) ⇒ (A ⇒ B)
     par ⇒I sur (2)–(8)
```

C'est la **contraposée classique** : "si ne pas B entraîne ne pas A, alors A entraîne B". Le point clé est la ligne (7) où on conclut `B` parce que l'hypothèse `¬B` mène à une contradiction. Ça, c'est du **raisonnement par l'absurde (RAA)**, donc c'est bien du Thème 5 (logique classique), pas du Thème 4.

Exercice 17 – Question 2
Prouver : `¬(A ∧ B) ⇒ (¬A ∨ ¬B)`

```
(1)  montrons ¬(A ∧ B) ⇒ (¬A ∨ ¬B)
     ┌──────────────────────────────────────────────────────────┐
(2)  │ supposons h₁ : ¬(A ∧ B)                                  │
     │                                                          │
(3)  │ A ∨ ¬A                      (tiers exclu sur A)          │
     │   ┌──────────── cas A ──────────┐                        │
(4)  │   │ supposons h₂ : A            │                        │
(5)  │   │ supposons h₃ : B            │                        │
(6)  │   │ A ∧ B                       │                        │
     │   │   par ∧I avec (4), (5)      │                        │
(7)  │   │ ⊥                           │                        │
     │   │   par E¬ avec (2), (6)      │                        │
(8)  │   │ ¬B                          │                        │
     │   │   par ¬I sur (5)–(7)        │                        │
(9)  │   │ ¬A ∨ ¬B                     │                        │
     │   │   par ∨I₂ avec (8)          │                        │
     │   └─────────────────────────────┘                        │
     │   ┌──────────── cas ¬A ─────────┐                        │
(10) │   │ supposons h₄ : ¬A           │                        │
(11) │   │ ¬A ∨ ¬B                     │                        │
     │   │   par ∨I₁ avec (10)         │                        │
     │   └─────────────────────────────┘                        │
(12) │ ¬A ∨ ¬B                                                  │
     │   par ∨E avec (3), (4)–(9), (10)–(11)                    │
     └──────────────────────────────────────────────────────────┘
(13) ¬(A ∧ B) ⇒ (¬A ∨ ¬B)
     par ⇒I sur (2)–(12)
```

Cette direction du De Morgan `¬(A ∧ B) ⇒ (¬A ∨ ¬B)` n'est **pas intuitionniste** en général. Ici on l'a fait avec le **tiers exclu sur A**. Idée : si A est vrai, alors B est impossible (sinon A ∧ B), donc ¬B. Si A est faux, alors déjà ¬A.

Exercice 17 – Question 3
Prouver : `F3 = ¬(A ⇒ B) ⇒ (A ∧ ¬B)`

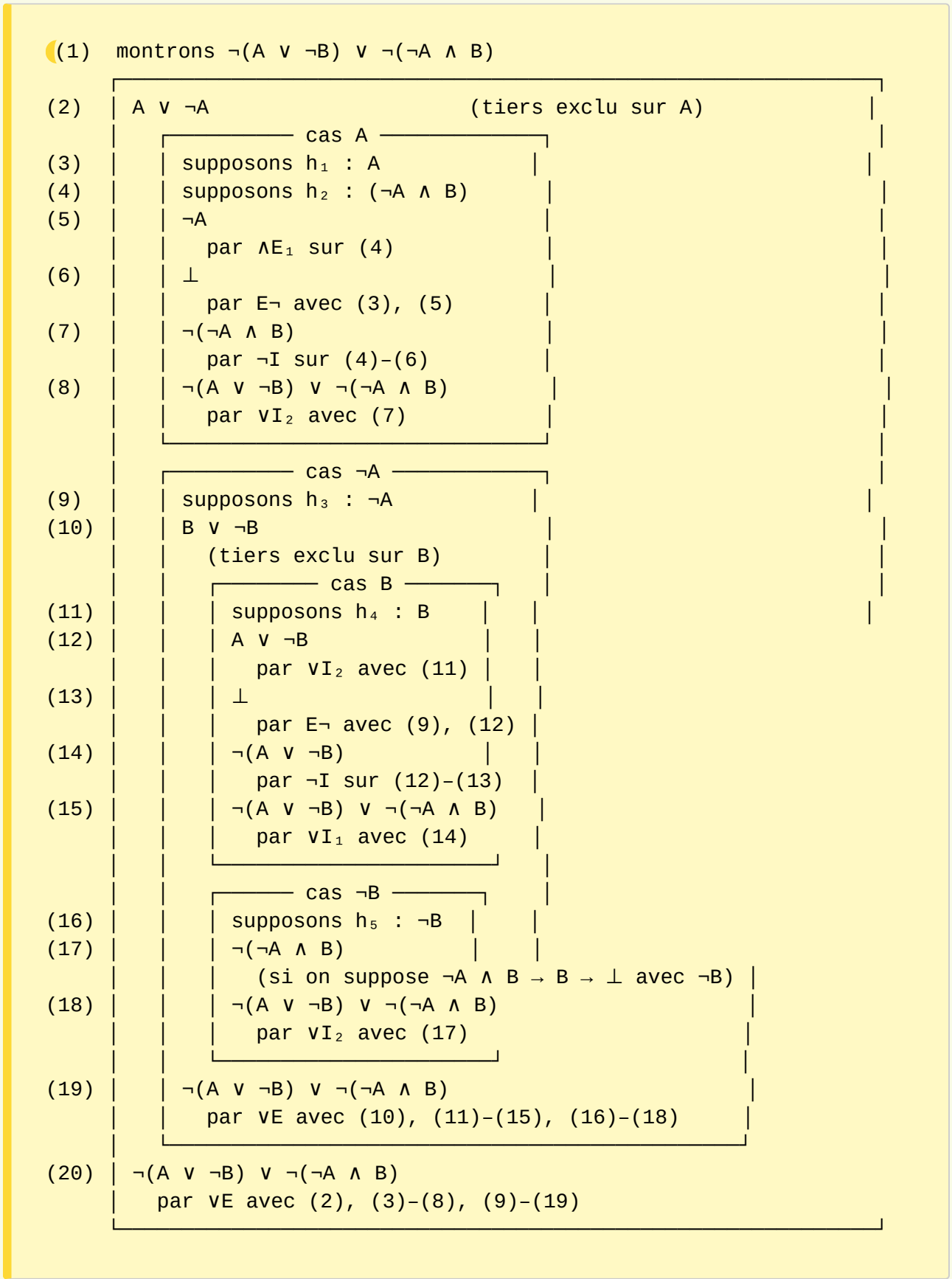
```
(1)  montrons ¬(A ⇒ B) ⇒ (A ∧ ¬B)
     ┌──────────────────────────────────────────────────┐
(2)  │ supposons h₁ : ¬(A ⇒ B)                          │
     │  -- objectif : montrer A et montrer ¬B           │
     │  -- d'abord A                                    │
     │   ┌──────────────────────────────────────────┐   │
(3)  │   │ supposons h₂ : ¬A                        │   │
(4)  │   │ A ⇒ B                                    │   │
     │   │   par ⇒I sur (3)–(?)                     │   │
     │   │   (explication : de A on aurait ⊥ donc B)    │
(5)  │   │ ⊥                                        │   │
     │   │   par E¬ avec (2), (4)                   │   │
     │   └──────────────────────────────────────────┘   │
(6)  │ A                                                │
     │   par RAA sur (3)–(5)                            │
     │  -- ensuite ¬B                                   │
     │   ┌──────────────────────────────────────────┐   │
(7)  │   │ supposons h₃ : B                         │   │
(8)  │   │ A ⇒ B                                    │   │
     │   │   par ⇒I sur (6)–(7)                     │   │
(9)  │   │ ⊥                                        │   │
     │   │   par E¬ avec (2), (8)                   │   │
     │   └──────────────────────────────────────────┘   │
(10) │ ¬B                                               │
     │   par ¬I sur (7)–(9)                             │
(11) │ A ∧ ¬B                                           │
     │   par ∧I avec (6), (10)                          │
     └──────────────────────────────────────────────────┘
(12) ¬(A ⇒ B) ⇒ (A ∧ ¬B)
     par ⇒I sur (2)–(11)
```

C'est la fameuse équivalence classique `¬(A ⇒ B) ≡ A ∧ ¬B`. On la fait "à la main" :

1. de `¬(A ⇒ B)` on tire d'abord **A** par RAA : si `¬A` alors `A ⇒ B` (contradiction), donc A ;
2. puis avec A on montre **¬B** : si B alors `A ⇒ B`, contradiction ;
3. on colle avec ∧I.

Tout ça utilise bien la logique classique (RAA).

Exercice 17 – Question 4
Prouver : `F4 = ¬(A ∨ ¬B) ∨ ¬(¬A ∧ B)`



Celle-là a l'air moche, mais l'idée est simple si tu la lis comme une tautologie : `¬(A ∨ ¬B) ∨ ¬(¬A ∧ B)` est équivalente à `(¬A ∧ B) ∨ (A ∨ ¬B)` (en développant les ¬ avec De Morgan). Et ça, c'est clairement toujours vrai : soit A, soit ¬B, soit (si A est faux et B vrai) le premier morceau.

En déduction naturelle stricte, on le fait avec 2 étages de **tiers exclu** : d'abord sur A, puis (dans le cas ¬A) sur B.

Exercice 18 – Enigme
On pose :

- `A` : "Anna est coupable"
- `J` : "Julie est coupable"

Les déclarations sont :

- Anna : « Julie est coupable. » → c'est la formule `J`
- Julie : « Si je suis coupable, alors Anna l'est aussi. » → c'est la formule `J ⇒ A`

Exercice 18 – Question 1
Montrer, par les règles de la déduction naturelle, que l'une au moins des deux déclarations est vraie. Autrement dit, montrer : `J ∨ (J ⇒ A)`.

```
(1)  J ∨ ¬J                    (tiers exclu sur J)
     ┌──────────── cas J ─────────────┐
(2)  │ supposons h₁ : J               │
(3)  │ J ∨ (J ⇒ A)                    │
     │   par ∨I₁ avec (2)             │
     └────────────────────────────────┘
     ┌──────────── cas ¬J ────────────┐
(4)  │ supposons h₂ : ¬J              │
(5)  │ supposons h₃ : J               │
(6)  │ ⊥                              │
     │   par E¬ avec (4), (5)         │
(7)  │ A                              │
     │   par ⊥E sur (6)               │
(8)  │ J ⇒ A                          │
     │   par ⇒I sur (5)-(7)           │
(9)  │ J ∨ (J ⇒ A)                    │
     │   par ∨I₂ avec (8)             │
     └────────────────────────────────┘
(10) J ∨ (J ⇒ A)
     par ∨E avec (1), (2)-(3), (4)-(9)
```

On a juste fait : “ou bien Julie est coupable → l’énoncé d’Anna est vrai, ou bien Julie ne l’est pas → alors ce qu’elle dit (J ⇒ A) est vrai.” C’est typiquement un raisonnement **classique** (on a utilisé `J ∨ ¬J`).

Exercice 18 – Question 2
On suppose maintenant :
– les innocents disent la vérité ;
– les coupables mentent.
Montrer qu’ **exactement une** des deux est coupable.

> On formalise d'abord les règles "dire vrai / mentir" pour Anna :

(1) ¬A ⇒ J (si Anna est innocente, elle dit vrai : "Julie est coupable") (2) A ⇒ ¬J (si Anna est coupable, elle ment : donc Julie n'est pas coupable)

On veut montrer : (A ∧ ¬J) ∨ (J ∧ ¬A)

(3) A ∨ ¬A (tiers exclu sur A) ┌──────── cas A ────────┐ (4) │ supposons h₁ : A │ (5) │ ¬J │ │ par ⇒E avec (2), (4) │ (6) │ A ∧ ¬J │ │ par ∧I avec (4), (5) │ (7) │ (A ∧ ¬J) ∨ (J ∧ ¬A) │ │ par ∨I₁ avec (6) │ └──────────────────────────────────┘ ┌──────── cas ¬A ────────┐ (8) │ supposons h₂ : ¬A │ (9) │ J │ │ par ⇒E avec (1), (8) │ (10) │ J ∧ ¬A │ │ par ∧I avec (9), (8) │ (11) │ (A ∧ ¬J) ∨ (J ∧ ¬A) │ │ par ∨I₂ avec (10) │ └──────────────────────────────────┘ (12) (A ∧ ¬J) ∨ (J ∧ ¬A) par ∨E avec (3), (4)–(7), (8)–(11)

Ici on n’a même pas eu besoin de la phrase de Julie 🤭. Tout vient du fait que : **si Anna est coupable alors Julie ne l’est pas**, et **si Anna est innocente alors Julie l’est**. Donc forcément : soit (Anna coupable, Julie innocente), soit (Julie coupable, Anna innocente). C’est exactement : `(A ∧ ¬J) ∨ (J ∧ ¬A)`.

On a utilisé le tiers exclu sur `A` → donc c’est bien de la **logique classique** du Thème 5.